

EXPERIMENT – 01

AIM:

To create an AWS (Amazon Web Services) account and set up the initial configuration required to access cloud computing services.

DESCRIPTION:

AWS (Amazon Web Services) is a comprehensive and widely used cloud platform that provides a variety of computing, storage, networking, and security services. It enables users to deploy scalable applications, store large amounts of data, and perform computational tasks on demand. AWS operates on a pay-as-you-go pricing model and offers a **Free Tier** that allows users to explore services without incurring immediate costs. Creating an AWS account is the first step toward leveraging cloud computing for application development, hosting, and data management. Once an account is set up, users gain access to the **AWS Management Console**, which serves as the central interface for managing AWS resources and services.

PROCEDURE:

Step 1: Open AWS Signup Page

1. Open a web browser and go to [AWS Official Website](https://aws.amazon.com/).
2. Click on "Create an AWS Account" at the top right corner.

Step 2: Enter Account Information

3. Provide the required details:
 - **Email Address** (used for logging into AWS)
 - **AWS Account Name** (e.g., "MyCloudLab")
 - **Secure Password** (with at least 8 characters, including numbers, symbols, and uppercase letters)
4. Click **Continue** to proceed.

Step 3: Provide Contact and Personal Details

5. Choose the type of account:
 - **Personal Account** (for individual users)
 - **Professional Account** (for businesses and organizations)
6. Enter your full name, phone number, and address.
7. Agree to the **AWS Customer Agreement** and click **Continue**.

Step 4: Enter Billing Information

8. Enter **credit or debit card** details for identity verification. AWS will place a small refundable charge to confirm payment information.
9. Click **Verify and Continue**.

Step 5: Identity Verification

10. Choose the preferred method of verification:
 - **SMS Verification** (Enter the code sent to your mobile phone)
 - **Voice Call Verification** (Follow the instructions provided in the call)
11. Once verified, click **Continue**.

Step 6: Choose an AWS Support Plan

12. Select a support plan:
 - **Basic (Free)** – Recommended for new users
 - **Developer (\$29/month)** – For testing and development
 - **Business (\$100/month)** – For production workloads
13. Click **Continue** after selecting the plan.

Step 7: Access the AWS Management Console

14. Once registration is complete, AWS will send an activation email.
15. Open the **AWS Management Console** by going to [AWS Console](#).
16. Log in using the registered **email** and **password**.
17. After login, explore the dashboard and access AWS services such as EC2, S3, and IAM.

OUTPUT:

- Successfully created an AWS account.
- Access to the AWS Management Console for managing cloud resources.
- Ready to launch and configure AWS services.

CONCLUSION:

This experiment successfully demonstrated the process of creating an AWS account. The step-by-step registration process included providing personal details, billing information, and verifying identity. Once the account was activated, the AWS Management Console became accessible, allowing users to explore cloud computing services. With an active AWS account, users can begin utilizing AWS services for deploying applications, managing storage, and performing data analytics efficiently.

AIM:

To develop a **full-stack React application** integrated with **AWS Amplify** for user authentication, GraphQL API interaction, and Lambda function execution — enabling a scalable and serverless web app with cloud-hosted backend services.

DESCRIPTION:

This project demonstrates how to build a cloud-powered web application using **ReactJS** and **AWS Amplify**. It includes:

- **User Authentication** via Amazon Cognito.
- A **GraphQL API** using AWS AppSync to manage data.
- A **Lambda function** to process or store user data.
- **Hosting** through Amazon S3 and CloudFront.
- Frontend styled using Amplify's UI components.

PROCEDURE:

Install Amplify CLI:

npm install -g @aws-amplify/cli

```
PS C:\Users\Lenovo> npm install -g @aws-amplify/cli
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer supported
npm warn deprecated glob@7.2.3: Glob versions prior to v9 are no longer supported

added 47 packages in 59s

12 packages are looking for funding
  run `npm fund` for details
PS C:\Users\Lenovo>
```

Create React App:

npx create-react-app my-amplify-app

cd my-amplify-app

```
>> cd my-amplify-app
Need to install the following packages:
create-react-app@5.1.0
Ok to proceed? (y) y

create-react-app is deprecated.

You can find a list of up-to-date React frameworks on react.dev
For more info see:https://react.dev/link/cra

This error message will only be shown once per install.

Creating a new React app in C:\Users\Lenovo\my-amplify-app.

Installing packages. This might take a couple of minutes.
Installing react, react-dom, and react-scripts with cra-template...
```

Initialize Amplify:

amplify init

```
PS C:\Users\Lenovo\my-amplify-app> amplify init
⚠For new projects, we recommend starting with AWS Amplify Gen 2, our new code-first
✓ Do you want to continue with Amplify Gen 1? (y/N) · yes
✓ Why would you like to use Amplify Gen 1? · Prefer not to answer
Note: It is recommended to run this command from the root of your app directory
? Enter a name for the project myamplifyapp
The following configuration will be applied:

Project information
| Name: myamplifyapp
| Environment: dev
| Default editor: Visual Studio Code
| App type: javascript
| Javascript framework: react
| Source Directory Path: src
| Distribution Directory Path: build
| Build Command: npm.cmd run-script build
| Start Command: npm.cmd run-script start
```

Add Authentication (Cognito):

amplify add auth

```
PS C:\Users\Lenovo\my-amplify-app> amplify add auth
Using service: Cognito, provided by: awscloudformation

The current configured provider is Amazon Cognito.

Do you want to use the default authentication and security configuration? Default configuration
Warning: you will not be able to edit these selections.
How do you want users to be able to sign in? Email or Phone Number
Do you want to configure advanced settings? No, I am done.
✓ Successfully added auth resource myamplifyappe66155fe locally
```

Add GraphQL API (AppSync):

amplify add api

- Define schema in amplify/backend/api/<yourApi>/schema.graphql.

```
PS C:\Users\Lenovo\my-amplify-app> amplify add api
? Select from one of the below mentioned services: GraphQL
? Here is the GraphQL API that we will create. Select a setting to edit or continue Continue
? Choose a schema template: Single object with fields (e.g., "Todo" with ID, name, description)

⚠ WARNING: your GraphQL API currently allows public create, read, update, and delete access to

✅ GraphQL schema compiled successfully.

Edit your schema at C:\Users\Lenovo\my-amplify-app\amplify\backend\api\myamplifyapp\schema.graph
√ Do you want to edit the schema now? (Y/n) · no
✅ Successfully added resource myamplifyapp locally
```

Add Lambda Function:

amplify add function

- Implement logic in amplify/backend/function/<yourFunc>/src/index.js.

```
PS C:\Users\Lenovo\my-amplify-app> amplify add function
? Select which capability you want to add: Lambda function (serverless function)
? Provide an AWS Lambda function name: saveUserEmail
? Choose the runtime that you want to use: NodeJS
? Choose the function template that you want to use: Hello World

? Do you want to configure advanced settings? No
? Do you want to edit the local lambda function now? Yes
Edit the file in your editor: C:\Users\Lenovo\my-amplify-app\amplify\backend\function\saveUserEmail\src\index.js
? Press enter to continue
✅ Successfully added resource saveUserEmail locally.
```

Push to AWS:

amplify push

Install Amplify Libraries:

npm install aws-amplify @aws-amplify/ui-react

Configure Amplify in src/index.js:

```
import { Amplify } from 'aws-amplify';
import awsExports from './aws-exports';
Amplify.configure(awsExports);
```

Add Hosting & Deploy:

amplify add hosting

amplify publish

Start Local App:

npm start

OUTPUT:

- User can **sign up / sign in** with Cognito-authenticated accounts.
- Authenticated users can **query / mutate data** via GraphQL API.
- Lambda function can be **triggered from the frontend** (e.g., to save or process user emails).
- App is **live and hosted** via S3 & CloudFront (URL generated by Amplify).

CONCLUSION:

This project successfully demonstrates the integration of **ReactJS with AWS Amplify**, showcasing a complete serverless architecture for modern web apps. Using Amplify's CLI and libraries, we can rapidly build scalable applications with built-in support for authentication, APIs, functions, and hosting — reducing backend complexity and deployment overhead.

EXPERIMENT – 02

AIM:

To create an EC2 (Elastic Compute Cloud) instance in AWS and configure it for use as a virtual server.

DESCRIPTION:

Amazon EC2 (Elastic Compute Cloud) is a scalable cloud computing service that provides resizable virtual machines (instances) for running applications. It allows users to deploy applications, host websites, and run workloads on demand. EC2 instances can be customized based on the required CPU, memory, and storage configurations. This experiment involves launching an EC2 instance, configuring security groups, and accessing the instance using SSH.

PROCEDURE:

Step 1: Log in to AWS Management Console

1. Open a web browser and go to [AWS Console](#).
2. Log in with your **AWS account credentials**.

Step 2: Navigate to EC2 Service

3. In the AWS Management Console, search for **EC2** in the search bar.
4. Click on **EC2** to open the EC2 dashboard.

Step 3: Launch an EC2 Instance

5. Click on **Launch Instance**.
6. Provide a name for the instance (e.g., "MyFirstEC2").

Launch an instance Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags Info

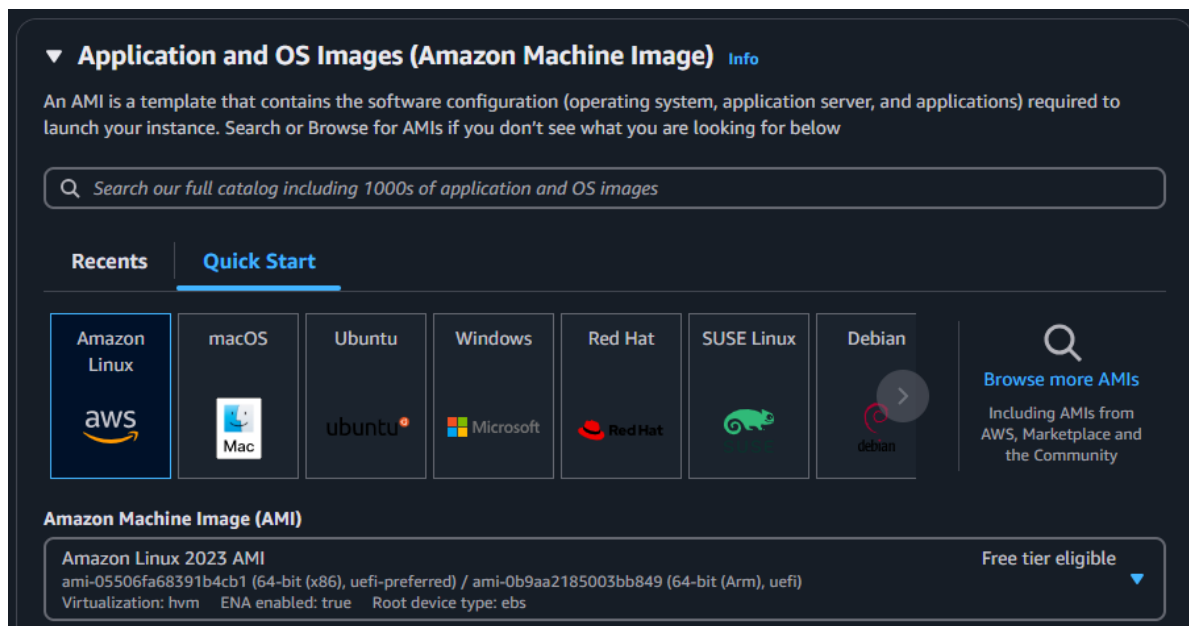
Name

MySecondEC2

[Add additional tags](#)

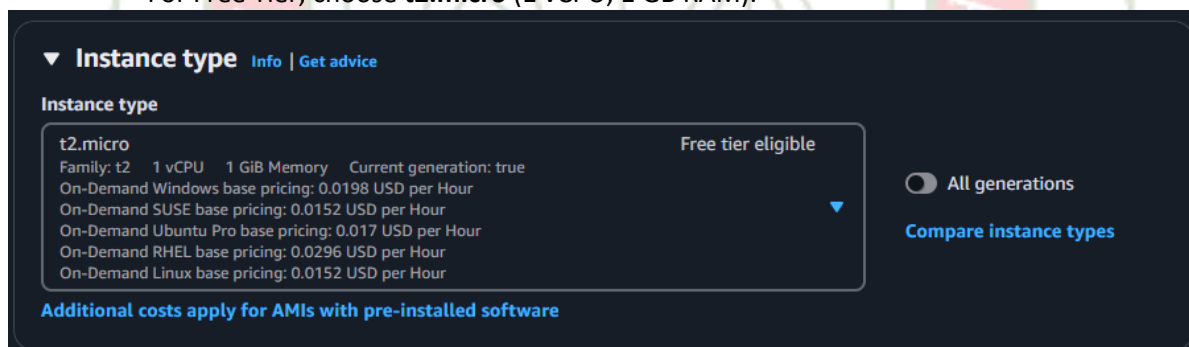
Step 4: Select an Amazon Machine Image (AMI)

7. Choose an **Amazon Machine Image (AMI)**.
 - **Amazon Linux 2** (Recommended for beginners)
 - **Ubuntu, Windows Server**, or other OS options are available.



Step 5: Choose an Instance Type

8. Select an **instance type** based on required CPU and RAM.
 - For Free Tier, choose **t2.micro** (1 vCPU, 1 GB RAM).



9. Click **Next: Configure Instance Details**.

Step 6: Configure Key Pair for Secure Access

10. Choose an existing key pair or create a new one:
 - Click Create new key pair.
 - Provide a name for the key pair.
 - Choose RSA as the key type.
 - Click Create key pair and download the .pem file.
 - Store it safely, as it will be used for SSH access.

Create key pair

✕

Key pair name

Key pairs allow you to connect to your instance securely.

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ RSA
RSA encrypted private and public key pair

☐ ED25519
ED25519 encrypted private and public key pair

Private key file format

☒ .pem
For use with OpenSSH

☐ .ppk
For use with PuTTY

⚠ When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. [Learn more](#) 

Cancel

Create key pair

Step 7: Configure Network and Security Group

11. **VPC Settings:** Use the default VPC.
12. **Subnet:** No preference (Default subnet in any availability zone).
13. **Auto-assign Public IP: Enable** (to allow external access).

▼ Network settings

Info

VPC - required

Info

vpc-0bd9334312eb1e959

(default) ▼

172.31.0.0/16

⌂

Subnet

Info

No preference ▼

⌂ Create new subnet

Auto-assign public IP

Info

Enable ▼

Additional charges apply when outside of free tier allowance

Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

14. **Security Group Settings:** Create a new security group (**launch-wizard-1**) with the following rules:
- **SSH (Port 22, Source: Anywhere - 0.0.0.0/0)** (Allows remote login)
 - **HTTP (Port 80, Source: Anywhere - 0.0.0.0/0)** (For web servers)
 - **HTTPS (Port 443, Source: Anywhere - 0.0.0.0/0)** (For secure web access)

Security group name - *required*

launch-wizard-2

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _-~/!@#,%&*~

Description - *required*

Info

launch-wizard-2 created 2025-03-29T11:35:27.623Z

Inbound Security Group Rules

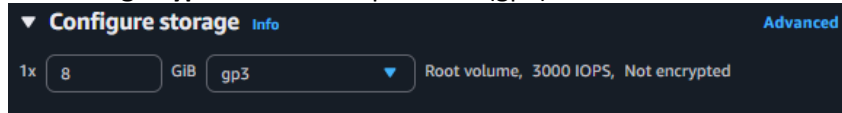
▼ Security group rule 1 (TCP, 22, 0.0.0.0/0)

Remove

Type	Info	Protocol	Info	Port range	Info
ssh	▼	TCP		22	
Source type	Info	Source	Info	Description - <i>optional</i>	Info
Anywhere	▼	Q Add CIDR, prefix list or sec.		e.g. SSH for admin desktop	
		0.0.0.0/0	✕		

Step 8: Configure Storage

15. Set the **Root Volume** size: **8 GiB (Free Tier Eligible)**.
16. **Storage Type**: General Purpose SSD (gp3).



Step 9: Review and Launch the Instance

17. Review the configuration summary:
 - **Instance Type**: t2.micro
 - **AMI**: Amazon Linux 2023
 - **Storage**: 8 GiB
 - **Security Group**: Allows SSH, HTTP, and HTTPS traffic
18. Click **Launch Instance**.

Step 10: Connect to the EC2 Instance

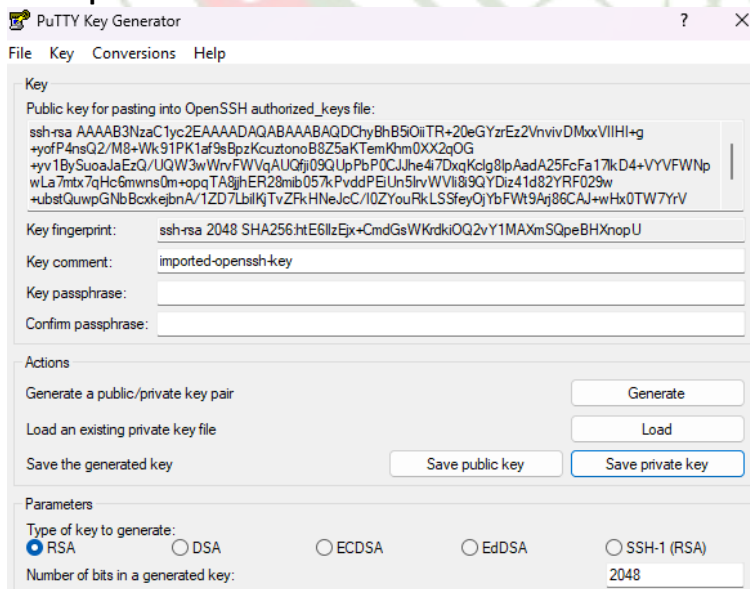
19. Once the instance is running, go to the **EC2 Dashboard > Instances**.
20. Select your instance and note the **Public IPv4 Address**.
21. Open a terminal (Linux/Mac) or use **PuTTY** (Windows).

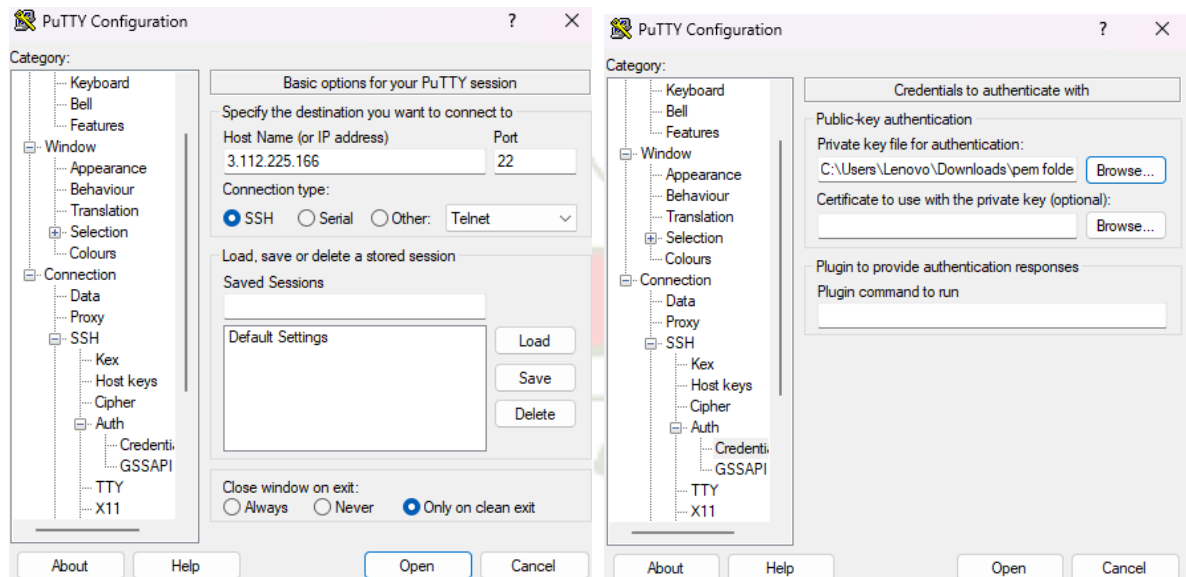
SSH Connection (Linux/macOS)

22. Run the following command:
Pgsq: `ssh -i /path/to/key.pem ec2-user@<public-ip>`
Example: `ssh -i MyFirstEC2.pem ec2-user@34.201.12.45`

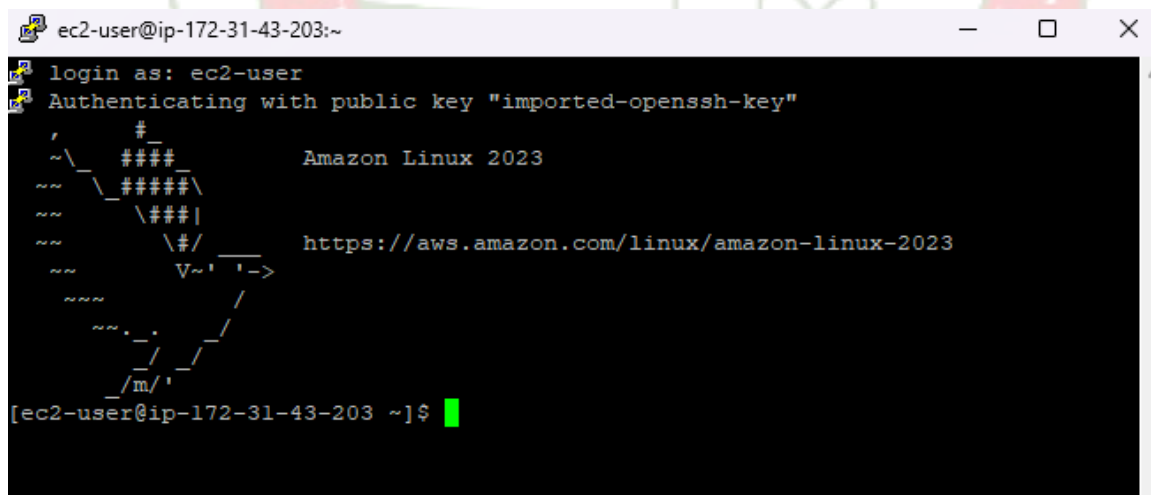
SSH Connection (Windows using PuTTY)

23. Convert the .pem file to .ppk using **PuTTYgen**.
24. Open **PuTTY** and enter the **Public IP** in the **Host Name** field.
25. Under **SSH > Auth**, browse for the .ppk key file.
26. Click **Open** to establish an SSH connection.





OUTPUT:



- Successfully launched an **EC2 instance** with Amazon Linux 2023 AMI.
- Configured storage, security groups, and network settings.
- Connected to the instance via SSH.

CONCLUSION:

This experiment successfully demonstrated the process of launching and configuring an **Amazon EC2 instance** in AWS. The instance was created using **Amazon Linux 2023 AMI**, configured with **t2.micro (Free Tier)**, and secured with a **key pair**. Additionally, we configured **security groups** to allow SSH, HTTP, and HTTPS traffic. After launching, we established a secure connection using **SSH**, enabling remote management of the instance. This setup provides the foundation for deploying cloud-based applications and services.

EXPERIMENT – 03

AIM:

To deploy a simple web application on an AWS EC2 instance and make it accessible via a public IP address.

DESCRIPTION:

Cloud computing provides the ability to deploy and run applications without needing to manage physical hardware. Amazon EC2 (Elastic Compute Cloud) allows users to create and configure virtual servers in the AWS cloud, making it a powerful platform for hosting applications.

In this experiment, we will deploy a **basic web application** on an EC2 instance using **Apache** as the web server. Apache is an open-source, widely used web server that enables hosting of web pages and applications. This experiment will guide us through:

- Launching an EC2 instance and connecting to it.
- Installing a web server inside the instance.
- Deploying a sample HTML-based web application.
- Accessing the application through a web browser.

By completing this experiment, we will understand how to use EC2 instances for hosting web applications and the fundamental steps required for cloud-based deployment. This experiment also introduces essential cloud concepts like **virtual servers, public IPs, security groups, and web hosting in AWS.**

PROCEDURE:

Step 1: Launch an EC2 Instance

Follow the steps from **Experiment 2** to create an EC2 instance with:

- **AMI (Amazon Machine Image):** Amazon Linux 2023 (or Ubuntu)
- **Instance Type:** t2.micro (Free-tier eligible)
- **Security Group Rules:**
 - Allow SSH (Port 22) – for connecting to the instance.
 - Allow HTTP (Port 80) – for web access.

Step 2: Connect to Your Instance

- **For Linux/Mac Users:** Open Terminal and connect via SSH:
`ssh -i your-key.pem ec2-user@your-public-ip`
- **For Windows Users (Using PuTTY):**
 - Open PuTTY.
 - Enter your **Public IP** in the "Host Name" field.
 - Under **Connection > SSH > Auth**, browse and select your **.ppk** key file.
 - Click **Open**, then log in with the username: `ec2-user`

Step 3: Install a Web Server (Apache)

Once connected to the instance via PuTTY or Terminal, install Apache:

For Amazon Linux:

```
sudo yum update -y
sudo yum install -y httpd
```

sudo systemctl start httpd
sudo systemctl enable httpd

```
[ec2-user@ip-172-31-43-203 ~]$ sudo yum update -y ; sudo yum install -y httpd ;  
sudo systemctl start httpd ; sudo systemctl enable httpd  
Last metadata expiration check: 0:03:05 ago on Sat Mar 29 12:44:49 2025.  
Dependencies resolved.  
Nothing to do.  
Complete!  
Last metadata expiration check: 0:03:06 ago on Sat Mar 29 12:44:49 2025.  
Package httpd-2.4.62-1.amzn2023.x86_64 is already installed.  
Dependencies resolved.  
Nothing to do.  
Complete!  
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr  
/lib/systemd/system/httpd.service.  
[ec2-user@ip-172-31-43-203 ~]$
```

For Ubuntu:

sudo apt update -y
sudo apt install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2

Check if the server is running:

sudo systemctl status httpd # For Amazon Linux
sudo systemctl status apache2 # For Ubuntu
If running, you will see active (running) in the output.

```
[ec2-user@ip-172-31-43-203 ~]$ sudo systemctl status httpd  
● httpd.service - The Apache HTTP Server  
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)  
   Active: active (running) since Sat 2025-03-29 12:47:55 UTC; 2min 52s ago  
     Docs: man:httpd.service(8)  
  Main PID: 27646 (httpd)  
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes  
      Tasks: 177 (limit: 1111)  
    Memory: 12.9M  
       CPU: 149ms  
    CGroup: /system.slice/httpd.service  
            └─27646 /usr/sbin/httpd -DFOREGROUND  
              └─27647 /usr/sbin/httpd -DFOREGROUND  
                └─27649 /usr/sbin/httpd -DFOREGROUND  
                  └─27650 /usr/sbin/httpd -DFOREGROUND  
                    └─27651 /usr/sbin/httpd -DFOREGROUND
```

Step 4: Deploy a Sample Web Application

1. Navigate to the web directory: *cd /var/www/html*
2. Create an **index.html** file: *sudo nano index.html*

```
[ec2-user@ip-172-31-43-203 ~]$ cd /var/www/html  
[ec2-user@ip-172-31-43-203 html]$ sudo nano index.html
```

3. Add the following HTML code:

```
ec2-user@ip-172-31-43-203:/var/www/html
GNU nano 8.3
!DOCTYPE html>
<html>
<head>
  <title>My EC2 Web App</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      text-align: center;
      margin: 0;
      padding: 0;
    }
    .container {
      width: 50%;
      margin: auto;
      background: white;
      padding: 20px;
      margin-top: 50px;
      border-radius: 10px;
      box-shadow: 0px 4px 8px rgba(0, 0, 0, 0.2);
    }
    h1 {
      color: #333;
    }
    p {
      font-size: 18px;
      color: #555;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>Welcome to My AWS EC2 Web Application!</h1>
    <p>This page is hosted on an Amazon EC2 instance.</p>
  </div>
</body>
</html>
```

4. Save the file (Ctrl + X → Y → Enter).

Step 5: Configure Firewall & Security Group and Restart the Web Server

Make sure your security group allows **HTTP (port 80)** access.

sudo systemctl restart httpd # For Amazon Linux

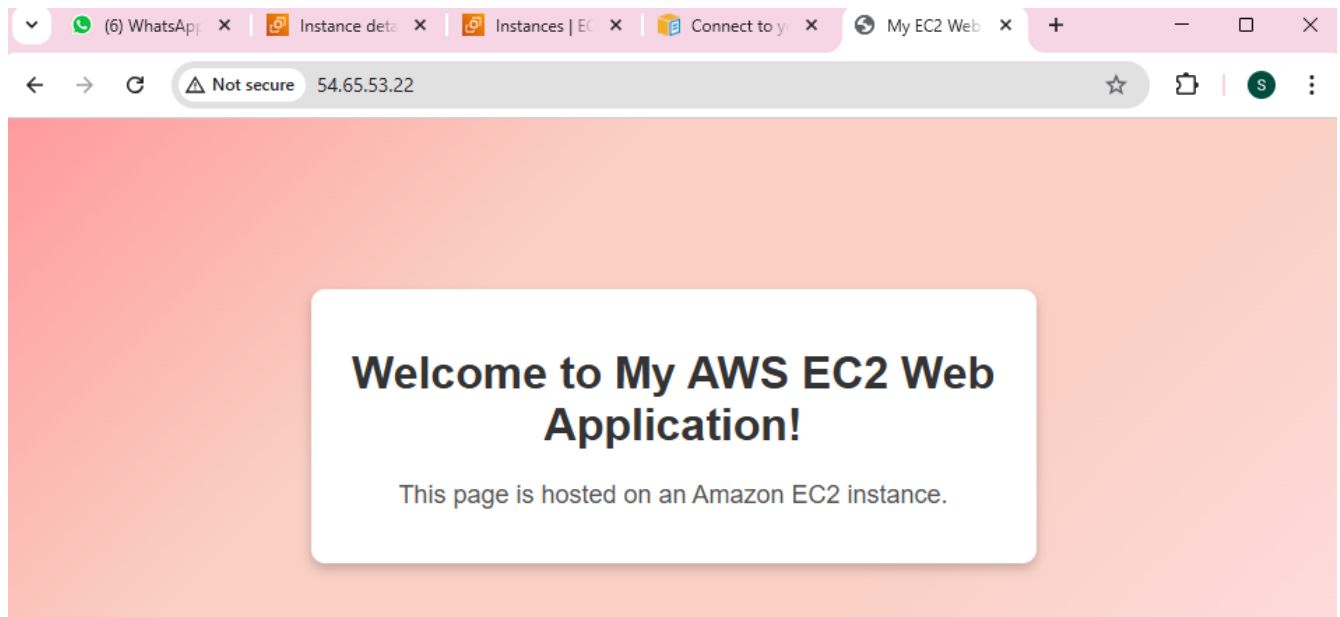
sudo systemctl restart apache2 # For Ubuntu

Step 6: Access the Application

Open a web browser and enter your EC2 **Public IP**: <http://your-public-ip>

If everything is set up correctly, you should see your **web application** displayed!

OUTPUT:



- Successfully deployed a web application on an EC2 instance.
- When accessing the **public IP**, the webpage is displayed with a **styled HTML page**.

CONCLUSION:

In this experiment, we successfully deployed a basic web application on an **Amazon EC2 instance** by installing and configuring an **Apache web server**. This experiment demonstrated key cloud computing concepts, including virtual machine provisioning, remote server access using SSH, web server installation, and security group configuration for network access.

By completing this experiment, we learned how to:

1. **Launch and configure an EC2 instance** – which provides scalable compute power without requiring physical hardware.
2. **Install and manage an Apache web server** – enabling the instance to serve web pages over the internet.
3. **Create and deploy a simple HTML web application** – demonstrating how cloud-based web hosting works.
4. **Understand basic Linux administration** – including working with package managers (yum or apt), modifying files using nano, and managing services (systemctl).

EXPERIMENT – 04

AIM:

To understand and implement virtualization concepts in an AWS environment by launching and managing virtual machines (instances) on Amazon EC2.

DESCRIPTION:

Virtualization is the process of creating virtual instances of computing resources such as servers, storage, and networks. AWS provides **Amazon EC2** (Elastic Compute Cloud) to enable users to create, manage, and run virtual machines (instances) efficiently. By using **hypervisors**, AWS allows multiple virtual instances to run on a single physical server, optimizing resource utilization.

In this experiment, we will:

- Understand the concept of **virtualization** and **hypervisors** used in AWS.
- Create and manage virtual machines (EC2 instances) using AWS.
- Configure and deploy a virtualized environment for running applications.

PROCEDURE:

Step 1: Log in to AWS Management Console

- Open [AWS Console](#).
- Sign in with your credentials.

Step 2: Launch an EC2 Instance (Virtual Machine)

- Go to **EC2 Dashboard**.
- Click **Launch Instance**.
- Choose an **Amazon Machine Image (AMI)** (e.g., Amazon Linux, Ubuntu).
- Select an **instance type** (e.g., t2.micro for free-tier usage).
- Configure the instance settings (default settings are fine for basic usage).
- Add storage as required.
- Create or select an **existing key pair** for SSH access.
- Configure **security groups** to allow required inbound/outbound traffic.
- Click **Launch Instance**.

Step 3: Connect to the Virtualized Instance

- Copy the **Public IP Address** of the instance.
- Open **PuTTY (Windows)** or **Terminal (Mac/Linux)**.
- Use the following command to connect via SSH:
`ssh -i your-key.pem ec2-user@your-public-ip`
- If using PuTTY, load the .ppk key file and enter the **Public IP Address**.

Step 4: Verify Virtualization

- Run the following command to check if virtualization is enabled:
`lscpu | grep Virtualization`
- If the output shows VT-x or AMD-V, virtualization is supported.
- Install additional packages if needed for containerized or nested virtualization environments.

```
Last login: Sat Mar 29 13:58:59 2025 from 103.44.0.250
[ec2-user@ip-172-31-43-203 ~]$ lscpu | grep Virtualization
Virtualization type: full
```

Step 5: Additional Packages for Nested Virtualization (KVM on AWS EC2)

- `sudo yum install -y qemu-kvm libvirt virt-install bridge-utils virt-viewer`

◆ Package Breakdown

Package	Purpose
qemu-kvm	The core package that enables KVM-based virtualization
libvirt	Provides an API to manage virtual machines
virt-install	A command-line tool to create virtual machines
bridge-utils	Helps configure virtual networking for VMs
virt-viewer	Provides a graphical interface to manage VMs

Step 6: Deploy a Virtualized Service

- Install **Docker** to run containerized applications within the virtualized EC2 instance:

```
sudo yum install docker -y
sudo systemctl start docker
sudo systemctl enable docker
```

```
[ec2-user@ip-172-31-43-203 ~]$ sudo yum install docker -y ; sudo systemctl start
docker ; sudo systemctl enable docker
Last metadata expiration check: 4:48:06 ago on Sat Mar 29 12:44:49 2025.
Dependencies resolved.
=====
Package                               Arch      Version                               Repository      Size
=====
Installing:
docker                                x86_64    25.0.8-1.amzn2023.0.1               amazonlinux     44 M
Installing dependencies:
containerd                            x86_64    1.7.25-1.amzn2023.0.1               amazonlinux     36 M
iptables-libs                         x86_64    1.8.8-3.amzn2023.0.2               amazonlinux     401 k
iptables-nft                          x86_64    1.8.8-3.amzn2023.0.2               amazonlinux     183 k
libcgroup                             x86_64    3.0-1.amzn2023.0.1                 amazonlinux     75 k
libnetfilter_conntrack                x86_64    1.0.8-2.amzn2023.0.2               amazonlinux     58 k
libnftnl                              x86_64    1.0.1-19.amzn2023.0.2              amazonlinux     30 k
libnftnl                              x86_64    1.2.2-2.amzn2023.0.2              amazonlinux     84 k
pigz                                  x86_64    2.5-1.amzn2023.0.3                 amazonlinux     83 k
runc                                   x86_64    1.2.4-1.amzn2023.0.1               amazonlinux     3.4 M
Transaction Summary
=====
```

- Verify Docker installation:

```
docker --version
```

```
[ec2-user@ip-172-31-43-203 ~]$ docker --version
Docker version 25.0.8, build 0bab007
```

- Run a sample container:

```
sudo docker run hello-world
```

```
[ec2-user@ip-172-31-43-203 ~]$ sudo docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

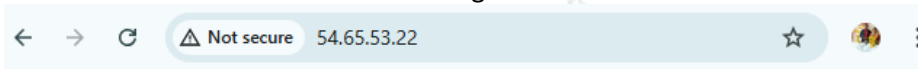
To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.
```

- Deploy a Web App in a Docker Container

`sudo docker run -d -p 80:80 nginx`

```
[ec2-user@ip-172-31-43-203 ~]$ sudo docker run -d -p 80:80 nginx
22b95e368449aa887b6c64d9a37b1850efa9960cf770095327d457e84ed407e5
```

- Verify the Deployment
Open a browser and enter `http://your-public-ip`
- You should see the NGINX Welcome Page.



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Step 7:

Required Packages for Kubernetes (K8s)

If you are running **Kubernetes (EKS, Minikube, or K3s)** inside AWS:

`sudo yum install -y kubect1 kubeadm kubelet`

`sudo systemctl enable kubelet && sudo systemctl start kubelet`

- kubect1 → CLI tool to manage Kubernetes clusters
- kubeadm → Used to initialize and configure Kubernetes nodes
- kubelet → Ensures running pods and containers inside Kubernetes

Additional Packages for VMware on AWS

If you are running **VMware Cloud on AWS**, install:

`sudo yum install -y open-vm-tools`

- open-vm-tools → Helps manage VMware VMs running inside AWS

OUTPUT:

- Successfully launched and connected to a virtualized EC2 instance.
- Verified the presence of virtualization support.
- Optionally deployed containerized applications using **Docker** to extend virtualization capabilities.

CONCLUSION:

This experiment demonstrated the implementation of virtualization in AWS through **Amazon EC2**. By launching and managing virtual machines, we explored **hypervisor-based virtualization**, resource allocation, and cloud infrastructure management.

Virtualization enabled efficient use of cloud resources.

AWS EC2 instances are virtual machines running on hypervisors.

Security groups and key pairs ensure secure instance access.

Containerization (Docker) can further optimize virtualization.

EXPERIMENT – 05

AIM:

To understand the concept of **Live Migration** in cloud computing and how AWS handles the live migration of virtual machines (EC2 instances) with minimal downtime.

DESCRIPTION:

Live migration refers to the process of moving a running virtual machine from one physical host to another with little or no interruption. While AWS does not provide a manual "Live Migration" feature for users like in traditional hypervisors (e.g., VMware), it automatically performs live migration in the background during maintenance events to ensure high availability and fault tolerance.

In this experiment, we will simulate the live migration concept by observing how AWS handles instance maintenance and attempts to migrate instances without stopping them.

PROCEDURE:

Step 1: Launch an EC2 Instance

- Go to AWS Management Console → EC2 → Launch an instance.
- Select a general-purpose AMI (e.g., Amazon Linux 2).
- Choose t3.micro or t3.small (these use **Nitro Hypervisor**, which supports live migration).
- Launch the instance and note the **Instance ID** and **Public IP**.

Step 2: Enable Monitoring

- Enable **Detailed Monitoring** (optional for deeper metrics).
- Use **CloudWatch** to track uptime and CPU/network activity during the experiment.

Step 3: Wait for AWS Scheduled Events (or simulate host degradation)

- Go to EC2 Dashboard → **Events** or open the instance → **Status Checks** → **Scheduled Events**.
- If AWS schedules a **system reboot** or **instance retirement**, AWS may attempt a live migration.
- During such events, AWS tries to move the instance **without stopping it**.

Step 4: Monitor the Instance During Migration

- Continuously ping the instance from a local terminal: `ping <EC2-Public-IP>`

```
[ec2-user@ip-172-31-43-203 ~]$ ping 13.230.53.190
PING 13.230.53.190 (13.230.53.190) 56(84) bytes of data.
^C
--- 13.230.53.190 ping statistics ---
217 packets transmitted, 0 received, 100% packet loss, time 224635ms
```

- Connect via SSH and run a continuous task:
`while true; do echo "Live migration test at $(date)"; sleep 5; done`
- Observe that the SSH session is **not interrupted**, indicating live migration.

Step 5: Check System Logs

- Run the following after suspected migration: `dmesg | grep -i migration`
Uptime

- You may see messages indicating host changes or memory state transfers.

Instances (1/4) [Info](#)

All states ▾

Last updated 3 minutes ago

Connect

Instance state ▾

Actions ▾

Launch instances ▾

☐	Name ↗ ▾	Instance ID	Instance state ▾	Instance type ▾	Status check	Alarm status	Availability Zone ▾	Public IPv4 DNS ▾	Public IPv4 ... ▾	Elastic IF
☐	MyFirstEC2	i-0fbe09045a684efa8	⊖ Stopped 🔍 🔍	t2.micro	–	View alarms +	ap-northeast-1a	–	–	–
☐	Instance 2	i-00b619cdf935e56ae3	⊖ Stopped 🔍 🔍	t2.micro	–	View alarms +	ap-northeast-1a	–	–	–
☐	Instance 1	i-0e4b414cfa8588086	⊖ Stopped 🔍 🔍	t2.micro	–	View alarms +	ap-northeast-1a	–	–	–
☒	MySecondEC2	i-05b051cc1081645cd	⊕ Running 🔍 🔍	t2.micro	⊕ 2/2 checks passed	View alarms +	ap-northeast-1a	ec2-54-65-42-13.ap-no...	54.65.42.13	–

- Instance launched and remained operational during AWS system events.
- Pings and SSH sessions were not interrupted.
- Logs showed no restart or downtime.
- Verified that AWS performed live migration in the background (if scheduled events occurred).

AWS does support **true live migration** of EC2 instances **internally**, particularly on **Nitro-based instances**. This live migration is **automatic, seamless, and transparent** to the user, designed to maintain uptime during infrastructure changes or maintenance. Unlike traditional platforms where users can trigger live migration manually, AWS abstracts this process to enhance operational simplicity and availability.

C B I T

EXPERIMENT – 06

AIM:

To configure and deploy an Application Load Balancer (ALB) in AWS to distribute incoming traffic across multiple EC2 instances for improved availability and scalability.

DESCRIPTION:

Load balancing is an essential technique used in cloud computing to efficiently distribute incoming network traffic across multiple servers. This ensures high availability, fault tolerance, and better performance for web applications.

AWS provides **Elastic Load Balancing (ELB)**, which includes different types of load balancers:

1. **Application Load Balancer (ALB)** – Operates at Layer 7 (HTTP/HTTPS), allowing routing based on URL paths, hostnames, and query parameters.
2. **Network Load Balancer (NLB)** – Operates at Layer 4 (TCP/UDP), providing ultra-fast performance for high-throughput applications.
3. **Gateway Load Balancer (GLB)** – Used for traffic inspection, security appliances, and advanced network monitoring.

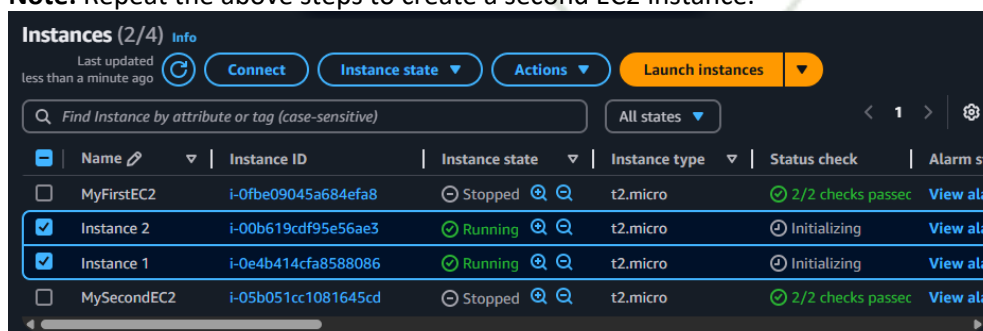
In this experiment, we focus on the **Application Load Balancer (ALB)**, which is ideal for **web applications** as it distributes HTTP/HTTPS traffic intelligently across multiple EC2 instances. The ALB enables automatic failover, meaning if one instance goes down, traffic is routed to healthy instances, ensuring **business continuity and reliability**.

PROCEDURE:

Step 1: Launch Two EC2 Instances

- Login to AWS Console.
- Navigate to **EC2 > Instances** and click **Launch Instance**.
- Select **Amazon Linux 2023 AMI**.
- Choose **t2.micro** (Free Tier Eligible).
- Configure:
 - **Auto-assign Public IP**: Enabled.
 - **Security Group**: Allow SSH (port 22) and HTTP (port 80).
- **Create a Key Pair** for SSH access.
- Click **Launch**.

Note: Repeat the above steps to create a second EC2 instance.



	Name	Instance ID	Instance state	Instance type	Status check	Alarm st
☐	MyFirstEC2	i-0f8e09045a684efa8	Stopped	t2.micro	2/2 checks passed	View ala
☒	Instance 2	i-00b619cdf95e56ae3	Running	t2.micro	Initializing	View ala
☒	Instance 1	i-0e4b414cfa8588086	Running	t2.micro	Initializing	View ala
☐	MySecondEC2	i-05b051cc1081645cd	Stopped	t2.micro	2/2 checks passed	View ala

Step 2: Install Web Server on Both Instances

- Connect to each EC2 instance using SSH or PuTTY.
- Run the following commands to install and start Apache:

```
sudo yum update -y
sudo yum install -y httpd
sudo systemctl start httpd
sudo systemctl enable httpd
```

```
[ec2-user@ip-172-31-33-173 ~]$ sudo yum update -y
sudo yum install -y httpd
sudo systemctl start httpd
sudo systemctl enable httpd

Amazon Linux 2023 Kernel Livepatch repository 105 kB/s | 15 kB 00:00
Dependencies resolved.
Nothing to do.
Complete!
Last metadata expiration check: 0:00:02 ago on Sun Mar 30 04:20:31 2025.
Dependencies resolved.
=====
Package Arch Version Repository Size
=====
Installing:
httpd x86_64 2.4.62-1.amzn2023 amazonlinux 48 k
Installing dependencies:
apr x86_64 1.7.5-1.amzn2023.0.4 amazonlinux 129 k
apr-util x86_64 1.6.3-1.amzn2023.0.1 amazonlinux 98 k
generic-logos-httpd noarch 18.0.0-12.amzn2023.0.3 amazonlinux 19 k
httpd-core x86_64 2.4.62-1.amzn2023 amazonlinux 1.4 M
=====
```

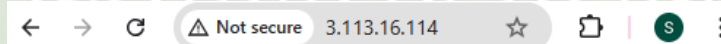
1. Add a simple HTML page to distinguish between instances:

- On Instance 1: `echo "<h1>Welcome to Server 1</h1>" | sudo tee /var/www/html/index.html`



Welcome to Server 1

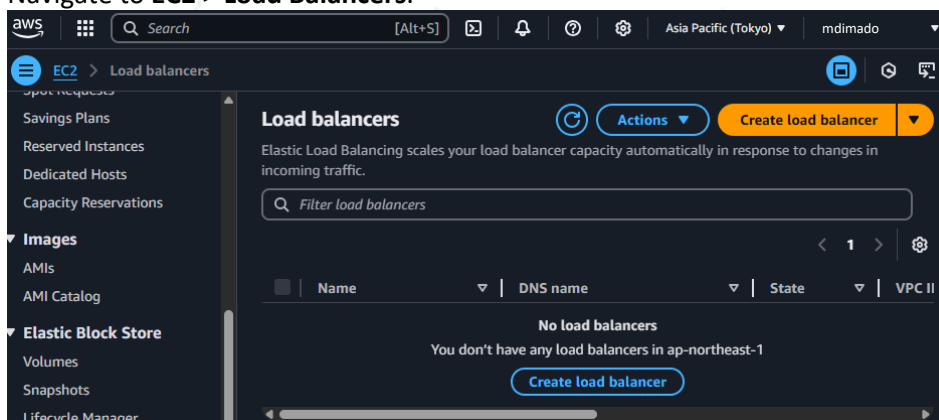
- On Instance 2: `echo "<h1>Welcome to Server 2</h1>" | sudo tee /var/www/html/index.html`



Welcome to Server 2

Step 3: Create an Application Load Balancer

- Navigate to EC2 > Load Balancers.



- Click **Create Load Balancer** > **Application Load Balancer**.
- Basic configuration:**

- Name: MyAppLoadBalancer

Basic configuration

Load balancer name
Name must be unique within your AWS account and can't be changed after the load balancer is created.

MyAppLoadBalancer

- Scheme: **Internet-facing**

Scheme | **Info**
Scheme can't be changed after the load balancer is created.

☒ **Internet-facing**

- Serves internet-facing traffic.
- Has public IP addresses.
- DNS name is publicly resolvable.
- Requires a public subnet.

☐ **Internal**

- Serves internal traffic.
- Has private IP addresses.
- DNS name is publicly resolvable.
- Compatible with the **IPv4** and **Dualstack** IP address types.

- IP Address Type:** Choose **IPv4**.

Load balancer IP address type | **Info**
Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP types. Public IPv4 addresses have an additional cost.

☒ **IPv4**
Includes only IPv4 addresses.

☐ **Dualstack**
Includes IPv4 and IPv6 addresses.

☐ **Dualstack without public IPv4**
Includes a public IPv6 address, and private IPv4 and IPv6 addresses. Compatible with **internet-facing** load balancers only.

- Configure Network Mapping:**

- VPC:** Select your existing **VPC** (vpc-0bd9334312eb1e959).

VPC | **Info**
The load balancer will exist and scale within the selected VPC. The selected VPC is also where the load balancer targets must be hosted (e.g., EC2 instances, Amazon S3, Amazon Lambda or on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view [target groups](#). For a new VPC, [create a VPC](#).

vpc-0bd9334312eb1e959
IPv4 VPC CIDR: 172.31.0.0/16

- Availability Zones & Subnets:** Choose at least two subnets for high availability:

- subnet-07920d6ac19d6194e (ap-northeast-1a)
- subnet-0b1f5a6e64759eb54 (ap-northeast-1c)

Availability Zones and subnets | **Info**
Select at least two Availability Zones and a subnet for each zone. A load balancer node will be placed in each selected zone and will automatically scale in response to traffic. The load balancer routes traffic to targets in the selected Availability Zones only.

☒ **ap-northeast-1a (apne1-az4)**

Subnet
Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently.

subnet-07920d6ac19d6194e
IPv4 subnet CIDR: 172.31.32.0/20

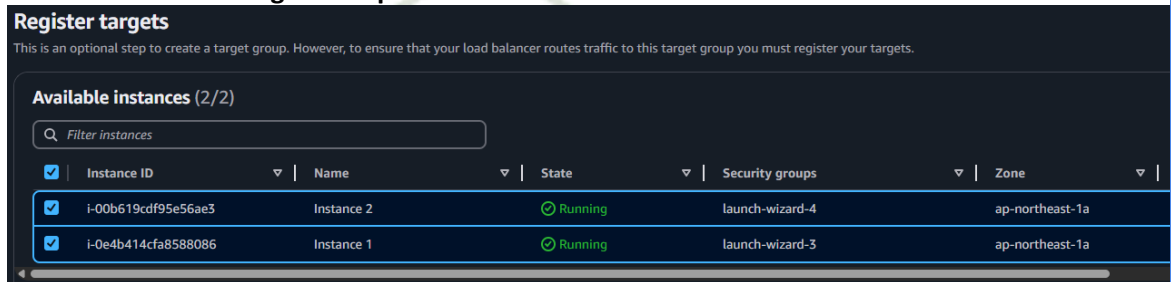
☒ **ap-northeast-1c (apne1-az1)**

Subnet
Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently.

subnet-0b1f5a6e64759eb54
IPv4 subnet CIDR: 172.31.0.0/20

☐ **ap-northeast-1d (apne1-az2)**

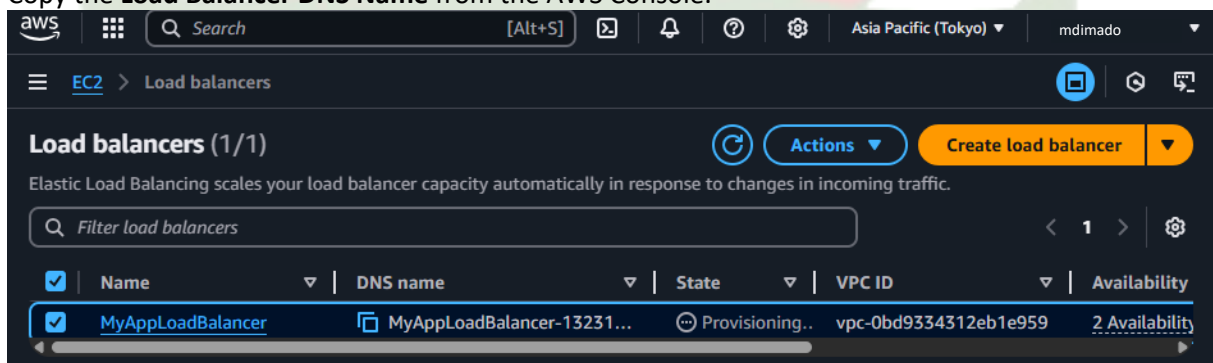
- **Configure Security Groups:**
 - Select an **existing security group** or create a new one.
 - Ensure it has **inbound rules** allowing:
 - **HTTP (Port 80)** from 0.0.0.0/0 (public access).
 - **HTTPS (Port 443)** if using SSL/TLS.
- **Configure Listeners:**
 - Add a **new listener** for **HTTP (Port 80)**.
 - Click **Forward to a Target Group**.



- **Create a Target Group:**
 - **Name:** MyTargetGroup.
 - **Target Type:** Select **Instances**.
 - **Protocol:** HTTP (Port 80).
 - **Health Check Protocol:** HTTP.
 - **Health Check Path:** /.
- **Register Targets:**
 - Select your **EC2 instances** (at least **two instances**).
 - **Enter 80** (for HTTP traffic).
 - Click **Include as Pending Below**.
 - Click **Create Target Group**.

Step 4: Test Load Balancer

- Copy the **Load Balancer DNS Name** from the AWS Console.



- Open a browser and enter: `http://<Load-Balancer-DNS>`
- Refresh the page multiple times. You should see responses from **both instances**, proving load balancing is working.

OUTPUT:

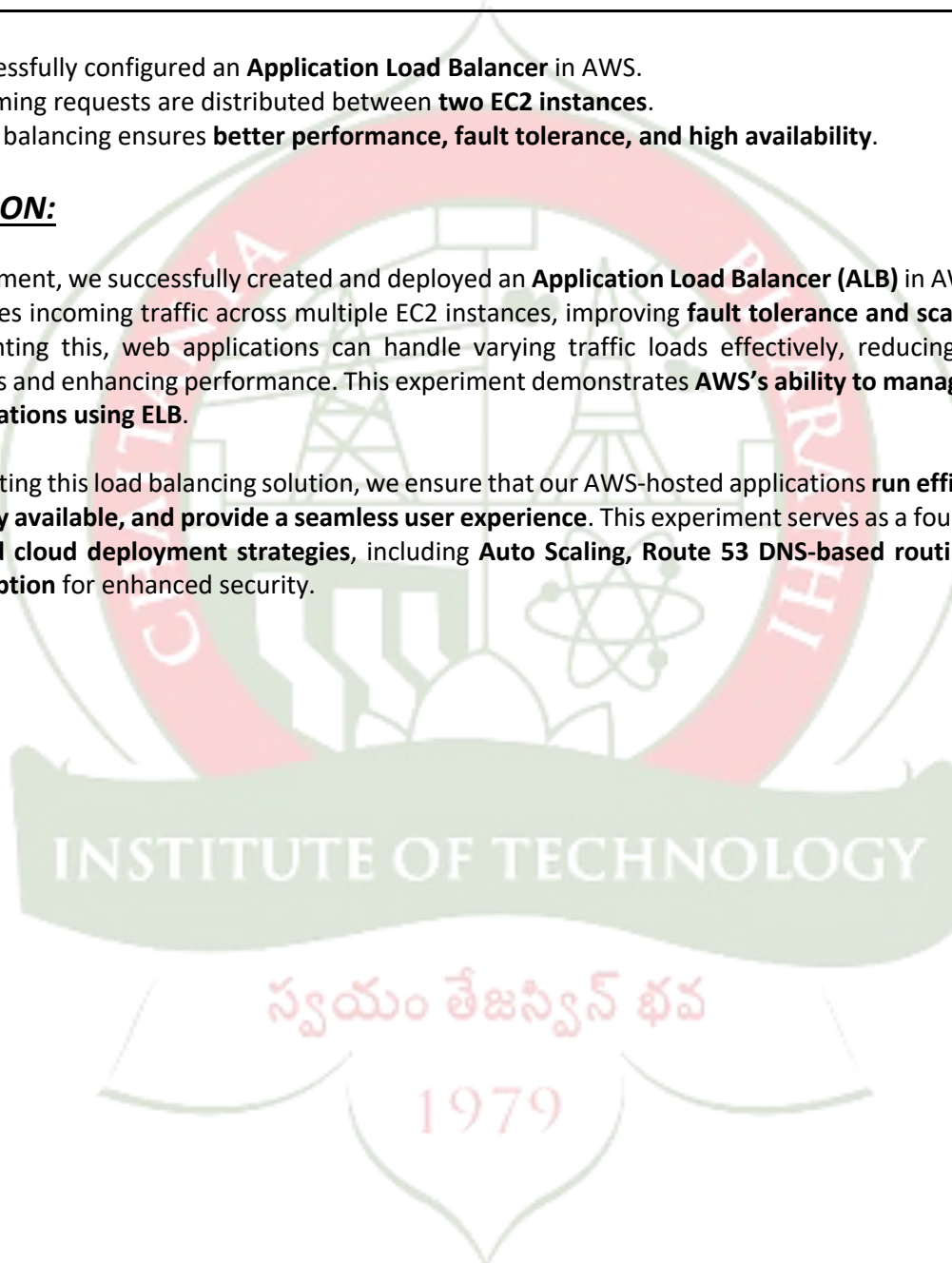


CONCLUSION:

CONCLUSION:

In this experiment, we successfully created and deployed an **Application Load Balancer (ALB)** in AWS. The ALB distributes incoming traffic across multiple EC2 instances, improving **fault tolerance and scalability**. By implementing this, web applications can handle varying traffic loads effectively, reducing server overload risks and enhancing performance. This experiment demonstrates **AWS's ability to manage high-traffic applications using ELB**.

By implementing this load balancing solution, we ensure that our AWS-hosted applications **run efficiently, remain highly available, and provide a seamless user experience**. This experiment serves as a foundation for **advanced cloud deployment strategies**, including **Auto Scaling, Route 53 DNS-based routing, and HTTPS encryption** for enhanced security.



EXPERIMENT – 07

AIM:

To perform database migration and implement CloudWatch monitoring in AWS.

DESCRIPTION:

1) Database Migration in AWS:

Database migration in AWS refers to moving data from one or more source databases to target databases hosted on AWS. This includes:

- On-premises to cloud migration
- Cloud-to-cloud transfers
- Conversion between different database engines

AWS provides the **AWS Database Migration Service (AWS DMS)** for efficient and secure migrations.

Key Concepts in AWS Database Migration:

- **Source and Target:** Original and destination databases.
- **Homogeneous vs Heterogeneous Migration:** Same database engine vs different database engines.
- **AWS DMS Features:** Minimal downtime, continuous replication, schema conversion support.
- **Components in DMS:** Replication instance, source/target endpoints, migration tasks.

Common Target Databases in AWS:

- Amazon RDS (MySQL, PostgreSQL, SQL Server, Oracle, MariaDB)
- Amazon Aurora
- Amazon Redshift
- Amazon DynamoDB (with some limitations)
- EC2-hosted custom databases

2) Amazon CloudWatch:

Amazon CloudWatch is a monitoring and observability service for AWS resources and applications.

It enables:

- Metrics collection and visualization
- Log storage and analysis
- Alarms and automated responses based on thresholds
- Detection of anomalies and optimization of resource usage

It supports integration with services like EC2, Lambda, RDS, ECS, and others.

PROCEDURE:

1) DATABASE MIGRATION

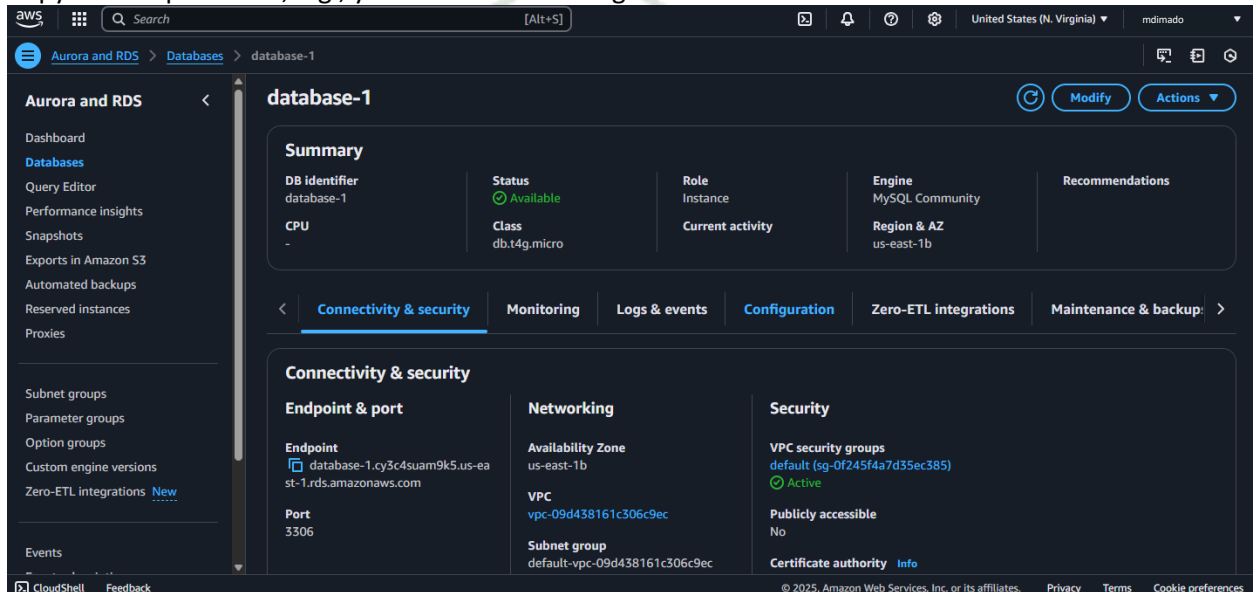
Step 1: Create an RDS Instance

- Go to **AWS Management Console** → **RDS**.
- Click **Create Database**.
- Choose:
 - Engine: MySQL or PostgreSQL
 - Template: Free Tier
 - DB Instance: db.t3.micro

- Set:
 - Username: admin
 - Password: yourpassword
- Enable **Public Access**.
- Configure **VPC Security Group** to allow inbound connections (port 3306 or 5432).
- Click **Create Database**.

Step 2: Get RDS Endpoint

- Go to **RDS Dashboard** → Your Database.
- Copy the Endpoint URL, e.g., your-db-name.abcdefg12345.us-east1.rds.amazonaws.com.



Step 3: Install MySQL and MySQL Workbench

- Download and install:
 - [MySQL Community Server](#)
 - [MySQL Workbench](#)
- Add MySQL to the **System PATH**.
 1. Copy the path to the bin folder, e.g.: C:\Program Files\MySQL\MySQL Server 8.0\bin
 2. Press Win + S and search for: Environment Variables → Click on "Edit the system environment variables"
 3. Click on "Environment Variables..."
 4. In the System variables section:
 - Select Path → Click Edit → Click New
 - Paste the path: C:\Program Files\MySQL\MySQL Server 8.0\bin
 - Click OK on all windows
- Verify installation by running: mysql -V

Step 4: Create Local Database

- Open MySQL terminal: mysql -u root -p
- Create a local database:

```
CREATE DATABASE localdb;  
USE localdb;  
CREATE TABLE test (  
  id INT,  
  name VARCHAR(50)  
);  
INSERT INTO test VALUES (1, 'Test User');
```

```
mysql> CREATE DATABASE localdb;
Query OK, 1 row affected (0.208 sec)

mysql> USE localdb;
Database changed
mysql> CREATE TABLE test (
  ->   id INT,
  ->   name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.405 sec)

mysql>
mysql> INSERT INTO test VALUES (1, 'PAVI');
Query OK, 1 row affected (0.181 sec)

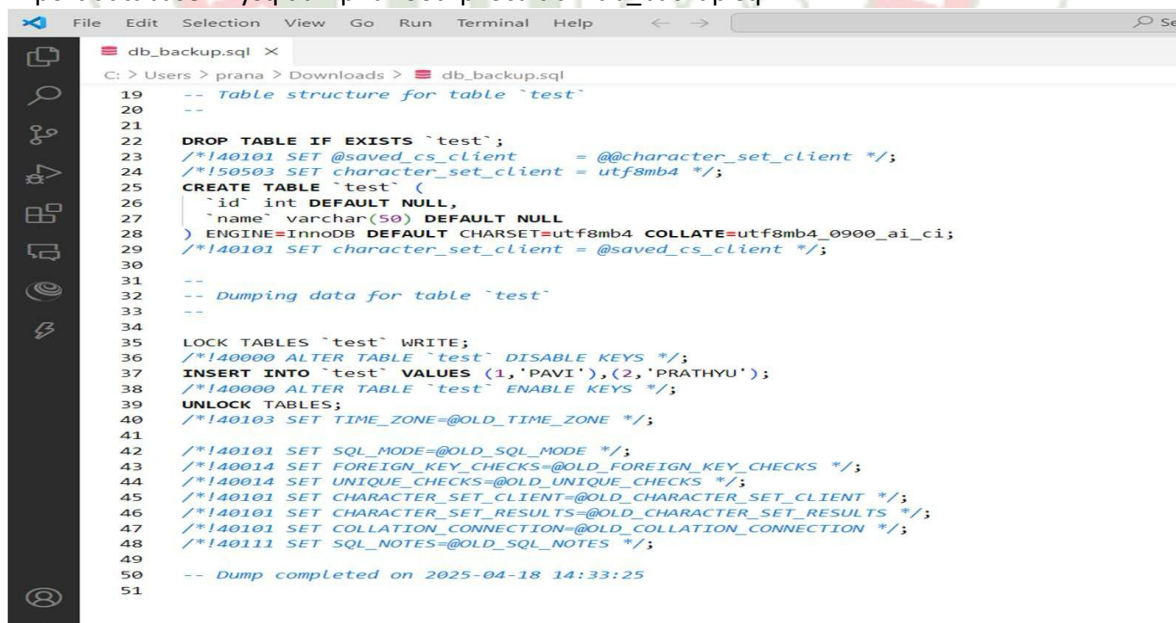
mysql> INSERT INTO test VALUES (2, 'PRATHYU');
Query OK, 1 row affected (0.159 sec)

mysql> exit;
Bye
```

- Verify data:
C:\Users\prana>mysql -u root -p
Enter password: *****
mysql> USE localdb;
mysql> SHOW TABLES;
mysql> SELECT * FROM test;
mysql> exit;

Step 5: Export Local Database (Backup)

- Exit MySQL shell.
- Export database: mysqldump -u root -p localdb > db_backup.sql



```
File Edit Selection View Go Run Terminal Help
db_backup.sql X
C:\Users\prana\Downloads > db_backup.sql
19 -- Table structure for table `test`
20 --
21
22 DROP TABLE IF EXISTS `test`;
23 /*140101 SET @saved_cs_client          = @@character_set_client */;
24 /*150503 SET character_set_client     = utf8mb4 */;
25 CREATE TABLE `test` (
26   `id` int DEFAULT NULL,
27   `name` varchar(50) DEFAULT NULL
28 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
29 /*140101 SET character_set_client     = @saved_cs_client */;
30
31 --
32 -- Dumping data for table `test`
33 --
34
35 LOCK TABLES `test` WRITE;
36 /*140000 ALTER TABLE `test` DISABLE KEYS */;
37 INSERT INTO `test` VALUES (1,'PAVI'),(2,'PRATHYU');
38 /*140000 ALTER TABLE `test` ENABLE KEYS */;
39 UNLOCK TABLES;
40 /*140103 SET TIME_ZONE=@OLD_TIME_ZONE */;
41
42 /*140101 SET SQL_MODE=@OLD_SQL_MODE */;
43 /*140014 SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS */;
44 /*140014 SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS */;
45 /*140101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
46 /*140101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
47 /*140101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
48 /*140111 SET SQL_NOTES=@OLD_SQL_NOTES */;
49
50 -- Dump completed on 2025-04-18 14:33:25
51
```

Step 6: Import Backup to AWS RDS

- Connect to RDS: mysql -h <RDS-endpoint> -u admin -p mydb
- Import the dump: source db_backup.sql

```
C:\Users\prana>mysql -h database-2.cz2cc224e1et.ap-south-1.rds.amazonaws.com -u admin4 -p mydb
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 44
Server version: 8.0.40 Source distribution

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Step 7: Verify the Data in RDS

- After connection:
SHOW TABLES;
SELECT * FROM test;
This will display the data from the test table in your RDS database.

```
C:\Users\prana>mysql -h database-2.cz2cc224e1et.ap-south-1.rds.amazonaws.com -u admin4 -p mydb < "C:\Users\prana\Downloads\db_backup.sql"
Enter password: *****

C:\Users\prana>mysql -h database-2.cz2cc224e1et.ap-south-1.rds.amazonaws.com -u admin4 -p mydb
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 54
Server version: 8.0.40 Source distribution

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

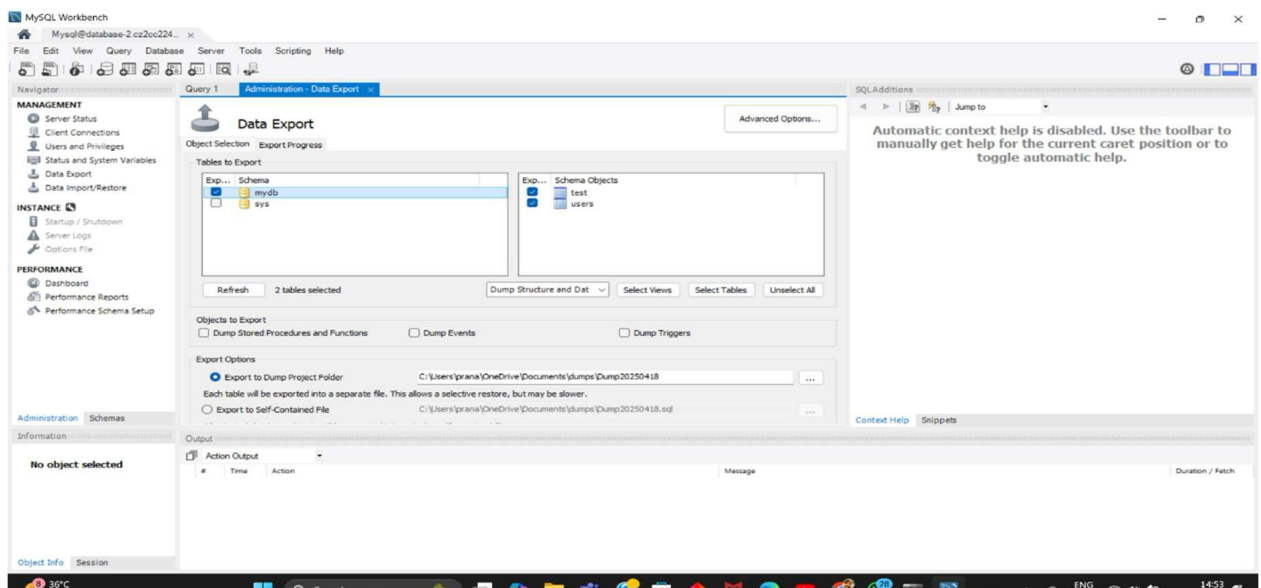
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW TABLES;
+-----+
| Tables_in_mydb |
+-----+
| test           |
| users          |
+-----+
2 rows in set (0.471 sec)

mysql> ^C
mysql> SELECT * FROM test;
+----+-----+
| id  | name |
+----+-----+
| 1   | PAVI |
| 2   | PRATHYU |
+----+-----+
2 rows in set (0.410 sec)
```

Step 8: Using MySQL Workbench (GUI Method)

- Open **MySQL Workbench**.
- Create a **New Connection**:
 - Connection Name: Any name (e.g., AWS RDS)
 - Hostname: database-2.cz2cc224e1et.ap-south-1.rds.amazonaws.com
 - Port: 3306
 - Username: admin4
 - Password: Click "Store in Vault..." and enter your password
- Click "Test Connection", then OK.
- Now click the connection to open it.
- In the left-side Schemas, select mydb and browse your tables under Tables.



2) CLOUDWATCH

Step 1: Install CloudWatch Agent on EC2 Instances

- SSH into EC2 instance:
`ssh -i your-key.pem ec2-user@your-ec2-ip`

Step 2: Go to CloudWatch Console

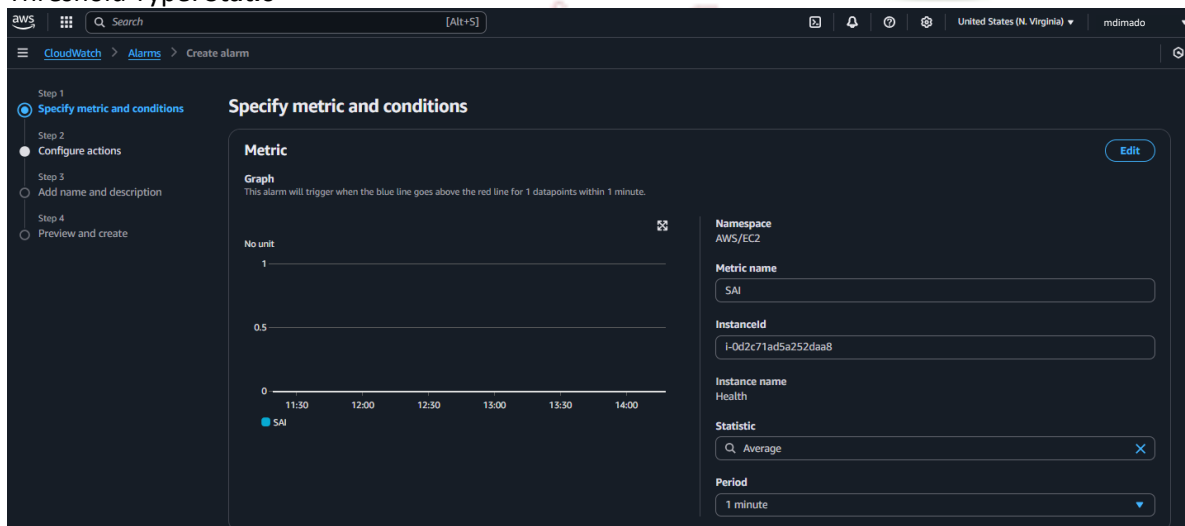
- Navigate to **CloudWatch > Alarms > Create Alarm**.

Step 3: Choose Metric

- Select Metric:
 - ELB > ApplicationELB > Per Load Balancer Metrics**
- Choose metrics such as:
 - Latency
 - HTTPCode_ELB_5XX_Count
 - RequestCount

Step 4: Define Alarm Conditions

- Statistic: **Average**
- Period: **1 minute** (or as needed)
- Threshold Type: **Static**



Step 5: Set Alarm Actions

- Configure the action when alarm triggers:
 - Send notification to **SNS topic** (email, SMS, or Lambda).

The screenshot shows the 'Configure actions' step in the AWS CloudWatch console. On the left, a sidebar lists steps: Step 1 (Specify metric and conditions), Step 2 (Configure actions - selected), Step 3 (Add name and description), Step 4 (Preview and create), and Step 5 (Preview and create). The main area is titled 'Configure actions' and contains three sections: 'Notification', 'Send a notification to the following SNS topic', and 'Email endpoints that will receive the notification...'. The 'Notification' section has three radio buttons: 'In alarm' (selected), 'OK', and 'Insufficient data'. The 'Send a notification to the following SNS topic' section has three radio buttons: 'Select an existing SNS topic', 'Create new topic' (selected), and 'Use topic ARN to notify other accounts'. The 'Create new topic' section has a text input field with the value 'Default_CloudWatch_Alarms_Topic'. The 'Email endpoints that will receive the notification...' section has a text input field with the value 'user@example.com'.

Step 6: Name and Create Alarm

- Give the alarm a meaningful name (e.g., HighLatencyAlarm-MyALB).
- Click **Create Alarm**.

The screenshot shows the 'Preview and create' step in the AWS CloudWatch console. On the left, a sidebar lists steps: Step 1 (Specify metric and conditions), Step 2 (Configure actions), Step 3 (Add name and description), Step 4 (Preview and create - selected), and Step 5 (Preview and create). The main area is titled 'Preview and create' and contains a section titled 'Step 1: Specify metric and conditions'. This section has a 'Metric' graph and a 'Details' panel. The 'Metric' graph shows a line chart with a blue line representing the metric 'SAI' and a red dashed line representing the threshold. The 'Details' panel contains the following information: Namespace: AWS/EC2, Metric name: SAI, InstanceId: i-0d2c71ad5a252daa8, Instance name: Health, Statistic: Average, and Period: 1 minute.

The screenshot shows the 'Alarms' page in the AWS CloudWatch console. The page has a sidebar with 'CloudWatch' and 'Alarms' selected. The main area shows a list of alarms. The table has columns: Name, State, Last state update (UTC), Conditions, and Act. The table contains one alarm named 'Falcon' with a state of 'Insufficient data' and a last state update of '2025-04-28 14:24:21'. The conditions are 'SAI >= 5 for 1 datapoints within 1 minute'. The 'Act' column shows a green checkmark.

Name	State	Last state update (UTC)	Conditions	Act
Falcon	Insufficient data	2025-04-28 14:24:21	SAI >= 5 for 1 datapoints within 1 minute	✓

CONCLUSION:

AWS data migration involves a structured process including assessment, planning, migration, and validation.

Key components of migration include AWS DMS for data transfer, Snowball for large-scale physical data movement, and DataSync for file transfers.

AWS Schema Conversion Tool (SCT) assists in heterogeneous database migrations, while **IAM roles** and **VPC settings** ensure secure migration.

Pre-migration testing prevents data loss and compatibility issues.

The process uses **staging tables**, **replication instances**, and **migration tasks** for structured data movement.

Amazon CloudWatch plays a vital role in:

- Monitoring AWS resources
- Setting alarms for abnormal conditions
- Optimizing application performance
- Reducing costs through real-time insights

By following AWS best practices, organizations can ensure smooth, secure, and efficient migration along with continuous system monitoring and management.



EXPERIMENT – 08

AIM:

To create AWS Lambda and Elastic Beanstalk.

DESCRIPTION:

AWS Lambda:

AWS Lambda is a serverless computing service that enables developers to run code in response to events without provisioning or managing servers. You simply upload your code as a function, set up a trigger (such as an HTTP request, S3 upload, or database update), and AWS handles the rest—scaling, patching, and running your code only when needed.

It supports multiple programming languages, including Python, Node.js, Java, and Go, making it highly versatile. Lambda is ideal for building event-driven applications, microservices, or automation scripts. It follows a pay-as-you-go model where you are charged based on the compute time your function consumes, making it a cost-effective and highly scalable solution for lightweight or short-duration tasks.

AWS Elastic Beanstalk:

AWS Elastic Beanstalk is a fully managed Platform-as-a-Service (PaaS) that simplifies the deployment and scaling of web applications and services. Developers can deploy applications by uploading their code, and Elastic Beanstalk automatically handles provisioning of infrastructure like EC2 instances, load balancers, auto-scaling groups, and monitoring tools.

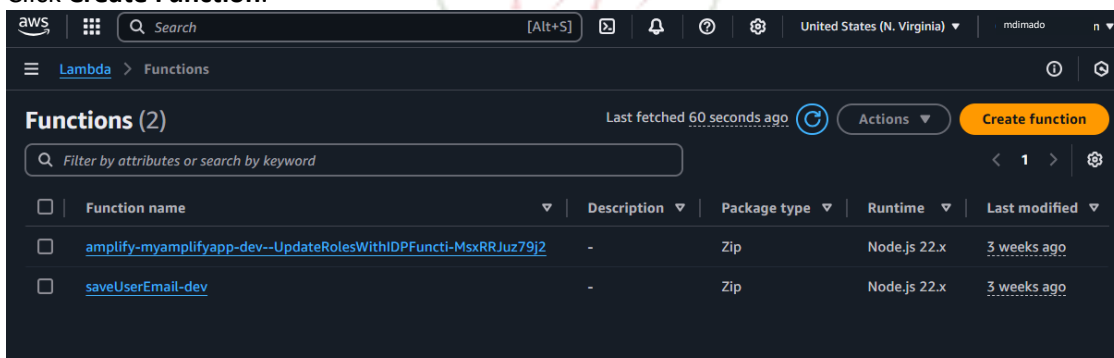
It supports several programming languages and platforms, including Java, Python, PHP, .NET, Node.js, Ruby, Go, and Docker containers. It is designed for developers who want to focus on writing code without worrying about infrastructure, offering built-in monitoring and flexibility to customize environments. It is a great choice for quickly launching and managing web applications.

PROCEDURE:

1) AWS Lambda:

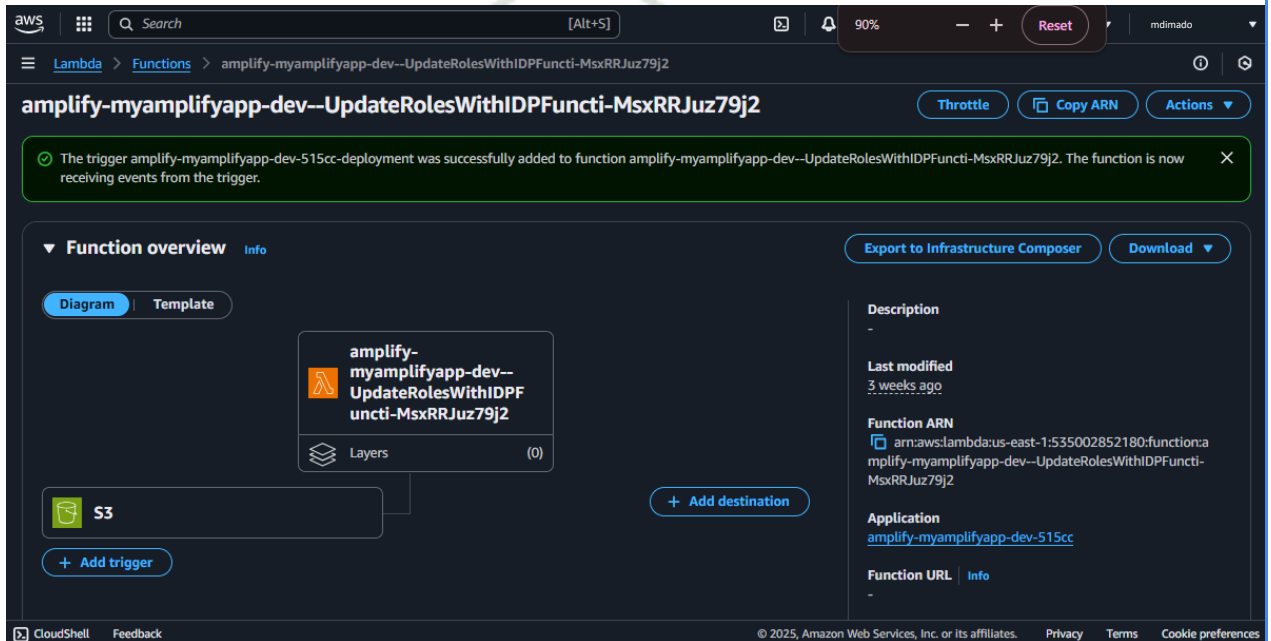
Step 1: Create a Lambda Function

- Open AWS Console → Search for "Lambda" → Click **Create Function**.
- Select:
 - **Author from scratch**
 - Give a function name (e.g., lambda-s3-function)
 - **Runtime:** Python 3.12
 - **Permissions:** "Create a new role with basic Lambda permissions"
- Click **Create Function**.



Step 2: Add an S3 Trigger

- Scroll down to "Function overview".
- Click "+ Add trigger".
- Select **S3** from the list.
- Configure:
 - **Bucket:** projectlambda001
 - **Event type:** All object create events
 - **Enable trigger** checkbox must be ticked.
- Click **Add**.



Step 3: Lambda Function Code (Python)

- Paste the following code inside the inline editor:

```
import json
import boto3
import urllib

def lambda_handler(event, context):
    s3_client = boto3.client('s3')
    bucket_name = event['Records'][0]['s3']['bucket']['name']
    key = event['Records'][0]['s3']['object']['key']
    key = urllib.parse.unquote_plus(key, encoding='utf-8')
    message = 'File ' + key + ' is successfully uploaded in bucket ' + bucket_name
    print(message)
    response = s3_client.get_object(Bucket=bucket_name, Key=key)
    content = response['Body'].read().decode()
    contents = json.loads(content)
    print("The data in the file is:\n", contents)
```

Step 4: Deploy the Code

- Click **Deploy** to save and activate the new code.

Step 5: Create & Upload a JSON File

- Open a text editor and paste the following sample JSON:

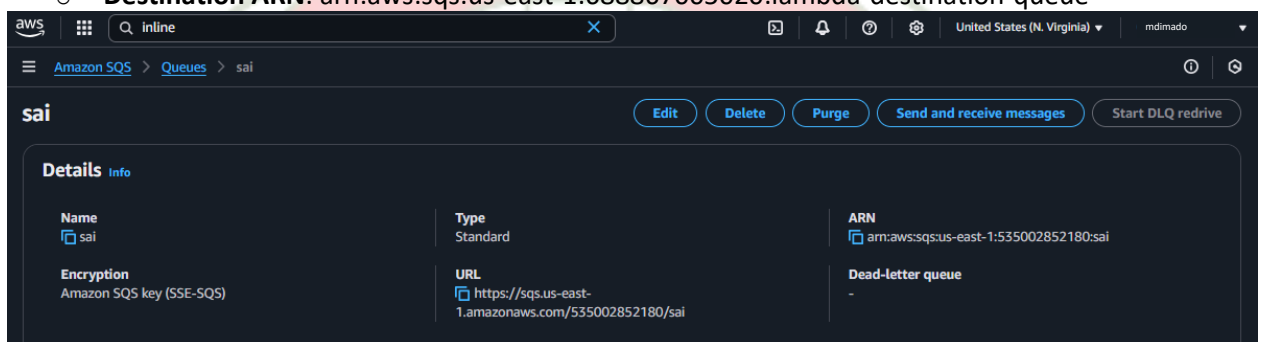
```
{
  "employee_id": "EMP001",
```

```
{
  "name": "Riya Sharma",
  "department": "Engineering",
  "skills": ["Python", "AWS", "DevOps"],
  "is_active": true,
  "experience_years": 4
}
```

- Save the file as sample.json.

Step 6: Set up SQS for Failures

- In case of function failure, AWS will send the failed event to an SQS queue.
- Search for **SQS** in the AWS console and create a new queue.
- Configure the Lambda function **"On failure"** settings:
 - **Destination type:** SQS
 - **Destination ARN:** arn:aws:sqs:us-east-1:688867005020:lambda-destination-queue

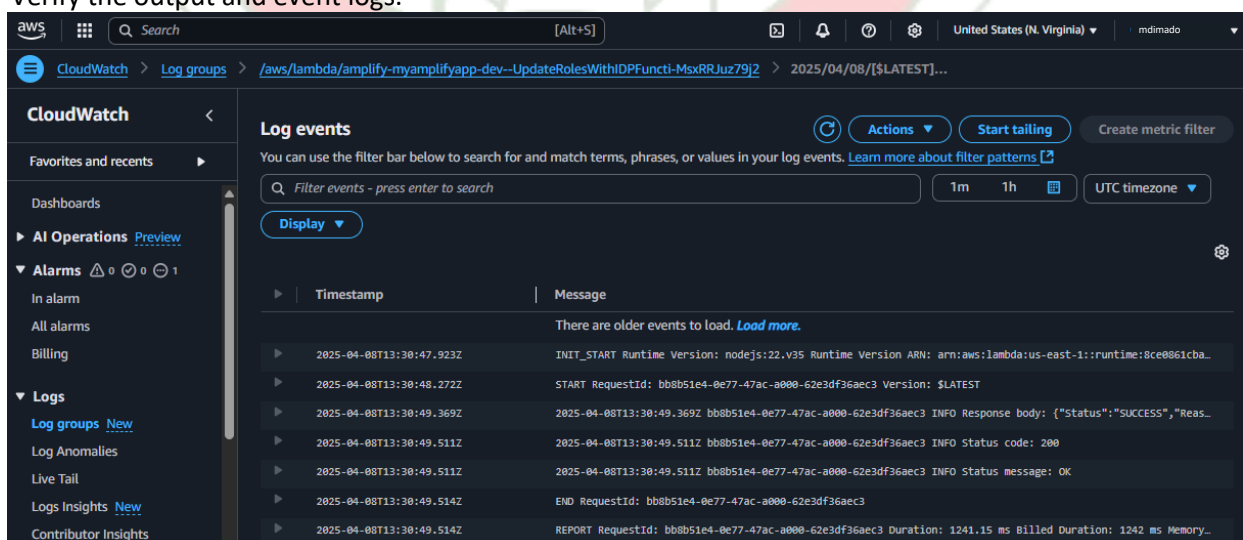


Step 7: Trigger Lambda Function

- Upload sample.json to the configured S3 bucket.
- Lambda will be triggered automatically.

Step 8: Check Logs

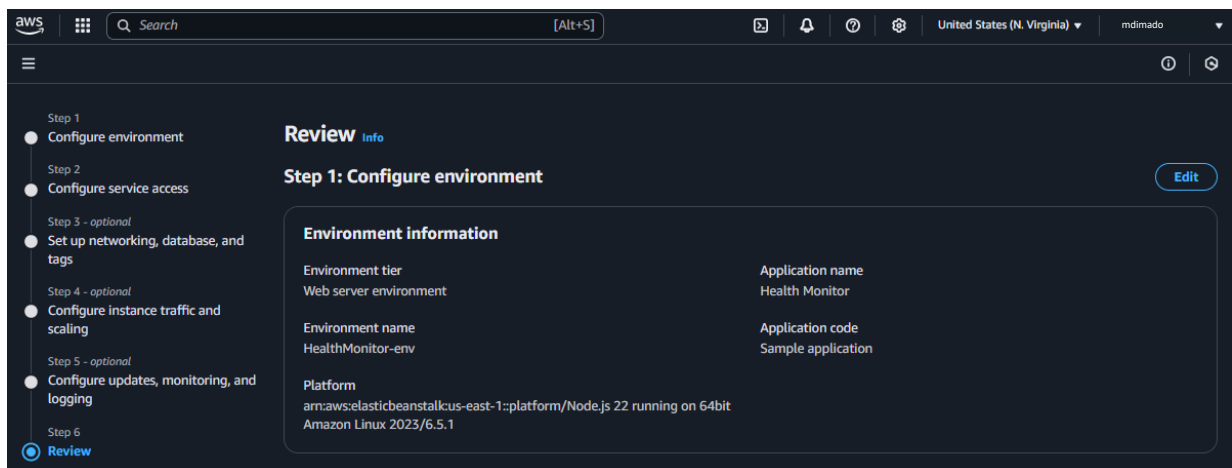
- Navigate to **Lambda > Monitor > View logs in CloudWatch**.
- Verify the output and event logs.



2) AWS Elastic Beanstalk:

Step 1: Create an Elastic Beanstalk Application

- Log in to the AWS Console → Navigate to **Elastic Beanstalk**.
- Click **Create Application**.



Step 2: Configure the Application

- Enter **Application Name** (e.g., php-sample-app).

Step 3: Choose Platform

- Under **Platform**:
 - **Platform**: PHP, **Platform branch**: PHP 8.2 running on 64bit Amazon Linux 2023.

Step 4: Application Code

- Select "**Sample application**" for code upload.

Step 5: Configure More Options

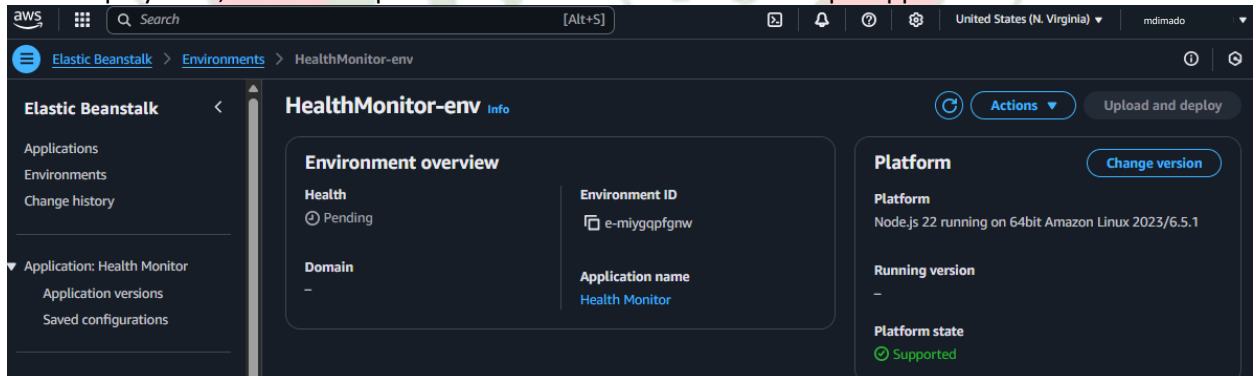
- Click **Configure more options**.
- Under **Configuration presets**, optionally select **High availability** for load-balanced environments.

Step 6: Deploy the Application

- Review all settings and click **Create app**.

Step 7: Access the Deployed Application

- After deployment, click on the provided domain URL to view the sample application live.



CONCLUSION:

AWS Lambda is a serverless, event-driven service ideal for running short-lived, stateless functions without the need to manage infrastructure. It provides automatic scaling, charges based on execution time, and integrates seamlessly with other AWS services, making it perfect for microservices, APIs, and event-driven processing. Elastic Beanstalk, on the other hand, simplifies the deployment of full-fledged web applications by automatically handling servers, scaling, and load balancing while offering customization capabilities.

Lambda is best suited for lightweight, on-demand tasks, whereas Beanstalk is designed for traditional, long-running applications.

Together, they offer flexible deployment options—**serverless agility with Lambda** and **managed convenience with Beanstalk**—allowing developers to choose based on their application requirements.

